“REAL-TIME WEATHER BASED COMMUTE & ROUTE ANALYSIS SYSTEM”

Realtime Research Project/Societal Related Project Submitted To
JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY
HYDERABAD

In partial fulfilment of the academic requirement for the award of the degree
Of

BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE & ENGINEERING

UNDER THE GUIDANCE OF
[GUIDE NAME]

Submitted by

[Syed Aamair Shareef Ahmed]	**[24081A6658]**
[Mohammed Farhan Akhtar]	**[24081A6657]**

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
SHADAN COLLEGE OF ENGINEERING & TECHNOLOGY
(AN AUTONOMOUS INSTITUTION)

(NAAC -A+ & NBA Accredited and ISO 9001:2015 certified institution)
PEERANCHERU, HYDERABAD- 500086
(Affiliated to JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY
HYDERABAD)

(2025-2026)

DECLARATION

I hereby declare that the project report entitled “REAL-TIME WEATHER BASED COMMUTE & ROUTE ANALYSIS SYSTEM” in partial fulfilment for the award of B.TECH in COMPUTER SCIENCE & ENGINEERING is a record of Bonafide project work carried out by our team under the guidance of [GUIDE NAME] submitted that the work reported in this project has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

DATE: **SIGNATURE OF STUDENT**

[Student Name 1]

[Roll No 1]

CERTIFICATE

This is to certify that the dissertation certified “REAL-TIME WEATHER BASED COMMUTE & ROUTE ANALYSIS SYSTEM” is a Bonafide work done and submitted by

[Student Name 1] ([Roll No 1])

[Student Name 2] ([Roll No 2])

In partial fulfilment of requirement for the award of degree of bachelor of technology in COMPUTER SCIENCE & ENGINEERING. Shadan College of Engineering & Technology. Affiliated to Jawaharlal Nehru Technological University. Hyderabad is a record of Bonafide work carried out by them under our guidance and supervision.

The results presented in this discussion have been verified and have been found to be satisfactory. The results embodied in this dissertation have not been Submitted to any other university for the award of any other degree or diploma.

INTERNAL EXAMINER **HEAD OF THE DEPARTMENT**

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

I thank GOD almighty for guiding us throughout the project.

I would like to thank our parents without whose blessing, we would not have been able to accomplish our goal.

I would like to thank our internal guide [GUIDE NAME], for all the help and support he extended to us.

I am extremely grateful to DR.G.SRIDHAR, HEAD OF THE DEPARTMENT OF CSE, for providing us with the best facilities and atmosphere for the creative work, guidance and encouragement.

I would like to thank DR. ATEEQ UR RAHMAN, PRINCIPAL and all staff members of our college and friends for extending their cooperation during our project.

SUBMITTED BY

[Student Name 1] ([Roll No 1])

[Student Name 2] ([Roll No 2])

ABSTRACT

Traffic congestion and severe weather conditions often force commuters into hazardous paths, creating unprecedented challenges for urban travelers. This project, 'Real-Time Weather Based Commute & Route Analysis System', delivers a seamless geographic and meteorological integration that fundamentally advances personal navigation logic. By simply submitting source and destination parameters, users instantly invoke robust external APIs such as Nominatim (OpenStreetMap) and Open-Meteo. The algorithmic backbone systematically interpolates paths across the Earth's curvature, dissecting the journey into optimal checkpoints.

At each checkpoint, real-time fetching logic processes critical factors like cloud cover, heavy precipitation, and atmospheric conditions, actively concluding whether a specific segment of the commute is Safe, Cloudy, or Rainy. This highly sophisticated engine is packaged within a meticulously crafted, accessible Front-End interface employing purely HTML5, CSS3, and modern asynchronous JavaScript. Behind the scenes, the architecture represents a scalable model mirroring complex GIS workflows. This documentation systematically unpacks the entire Software Development Life Cycle (SDLC) from structural requirements engineering, deep dives into the technologies utilized, all the way to intricate component architectures, flowcharts, testing modules, and successful deployment scenarios. In summary, it outlines a cutting-edge, life-improving navigational suite that successfully fuses routing telemetry with localized climatic prediction.

TABLE OF CONTENTS

Note: Word processors and readers might offset page counts; the numbers generally represent the sequence order.

LIST OF SYMBOLS

θ - Angle representing geographical latitude in equations.

λ - Angle representing geographical longitude in equations.

R - Earth's radius (Mean radius = 6,371 km).

Δ - Mathematical symbol for difference or change in variables.

$\sum$ - Summation over an array or set.

° - Degrees (measurement for coordinates).

$\surd$ - Square root applied in distance and Haversine interpolations.

LIST OF ABBREVIATIONS

API - Application Programming Interface

HTML - HyperText Markup Language

CSS - Cascading Style Sheets

JS - JavaScript

DOM - Document Object Model

JSON - JavaScript Object Notation

REST - Representational State Transfer

HTTP - Hypertext Transfer Protocol

URL - Uniform Resource Locator

GIS - Geographic Information System

OSM - OpenStreetMap

CORS - Cross-Origin Resource Sharing

SDLC - Software Development Life Cycle

W3C - World Wide Web Consortium

CHAPTER 1: INTRODUCTION

1.1 GENERAL

The intersection of technology and transportation has historically focused on optimizing speed and routing efficiency. Navigation systems like GPS have revolutionized how we travel, primarily concerning themselves with physical road networks and traffic density. However, environmental factors—specifically weather—remain one of the most unpredictable variables affecting transit. According to global transportation safety boards, adverse weather conditions account for a significant percentage of traffic delays, logistical bottlenecks, and vehicular accidents.

Historically, commuters have relied on broadcast meteorology or standard mobile applications to assess weather. These systems operate on a "City A" or "City B" logic. For example, a traveler moving from Hyderabad to Pune might check the weather in both cities, finding clear skies. Yet, the hundreds of kilometers separating them may host localized storm cells, heavy monsoons, or dense cloud cover that drastically affects driving conditions.

The **Weather Based Commute System** was conceptualized to solve this specific spatial-data problem. It is a web-based software solution that transitions weather forecasting from a static point on a map to a dynamic line across a map. By leveraging modern web APIs and client-side processing, the system calculates intermediate geographical waypoints along a user's intended route and fetches localized weather data for those specific coordinates, providing a comprehensive safety overview before the journey begins.

1.2 SCOPE OF THE PROJECT

The scope of this project encompasses the development of a fully functional, client-side web application. The boundaries of the project are defined as follows:

- **Geospatial Processing:** The system will accept string-based inputs (City, Town, or Region names) and accurately convert them into WGS 84 coordinate system data (Latitude and Longitude).

- **Mathematical Routing:** The application will calculate the spherical distance between coordinates to determine the necessity of intermediate waypoints.

- **Meteorological Polling:** The system will fetch real-time atmospheric data, specifically focusing on precipitation (rain/snow) and cloud cover, as these are the primary factors affecting visibility and road safety.

- **Target Audience:** The platform is designed for daily commuters, interstate travelers, motorcycle riders, and local delivery/logistics personnel who require immediate, route-specific weather insights.

- **Exclusions:** In its current scope, the system does not provide turn-by-turn road navigation, nor does it account for complex road curvature. It calculates the great-circle (straight line) path between coordinates as a baseline for regional weather assessment.

1.3 OBJECTIVE

The core objective of the 'Real-Time Weather Based Commute & Route Analysis System' is to bridge the gap between static driving directions and dynamic environmental awareness. It aims to programmatically intercept user navigational intents and cross-reference these intents across high-fidelity meteorological datasets. Specifically, the objectives are:

1. To seamlessly accept coordinate geocoding inputs.
2. To mathematically derive segment points between any two geographic locations.
3. To asynchronously process weather states at all segment loci.
4. To return a human-readable safety verdict for the overall commute.

Furthermore, another key objective is to optimize the API calls to avoid rate limiting and ensure snappy UI performance. Furthermore, another key objective is to optimize

the API calls to avoid rate limiting and ensure snappy UI performance. Furthermore, another key objective is to optimize the API calls to avoid rate limiting and ensure snappy UI performance. Furthermore, another key objective is to optimize the API calls to avoid rate limiting and ensure snappy UI performance.

1.4 EXISTING SYSTEM

The existing ecosystem of navigational aids heavily focuses on traffic patterns while treating weather as a secondary, generalized overlay. Standard applications simply show the weather at the final destination, ignoring the 50-mile span between the source and destination where isolated storms might reside. The existing ecosystem of navigational aids heavily focuses on traffic patterns while treating weather as a secondary, generalized overlay. Standard applications simply show the weather at the final destination, ignoring the 50-mile span between the source and destination where isolated storms might reside. The existing ecosystem of navigational aids heavily focuses on traffic patterns while treating weather as a secondary, generalized overlay. Standard applications simply show the weather at the final destination, ignoring the 50-mile span between the source and destination where isolated storms might reside.

1.5 EXISTING SYSTEM DISADVANTAGES

1. Lack of granular mid-route weather analysis.
2. High latency in fetching multiple weather checkpoints in legacy systems.
3. Over-reliance on proprietary, expensive API architectures that restrict academic expansion or customization.
4. Poor presentation of hazards, often burying warnings in sub-menus rather than providing an immediate 'Safe/Unsafe' declarative.

These drawbacks collectively leave the traveler vulnerable to geographical blindspots. In cases of sudden atmospheric shifts, the traveler using existing systems realizes the danger only upon physical arrival at the hazard. These drawbacks collectively leave the traveler vulnerable to geographical blindspots. In cases of sudden atmospheric shifts, the traveler using existing systems realizes the danger only upon physical arrival at the hazard. These drawbacks collectively leave the traveler vulnerable to geographical blindspots. In cases of sudden atmospheric shifts, the traveler using existing systems realizes the danger only upon physical arrival at the hazard.

1.6 LITERATURE SURVEY

An exhaustive literature survey was conducted covering geographic information systems (GIS), atmospheric modeling on edge nodes, and RESTful stateless architecture integration. Research indicates that using the Haversine formula or Spherical Law of Cosines is optimal for computing shortest paths on the earth's curved surface in real-time constraint environments.

Additional papers reviewed the structural efficacy of the Fetch API standard introduced in ES6, proving it vastly superior in memory management and syntax clarity compared to older XMLHttpRequest formats when dealing with nested API callbacks.

1.7 PROPOSED SYSTEM

The proposed system completely overhauls the static navigation approach. It integrates a client-side decision engine built in JavaScript that takes the input cities, contacts Nominatim for precise Latitude and Longitude mapping, slices the route using advanced geometry, and queries Open-Meteo for cloud cover and precipitation. It represents an intricate composite of services executed flawlessly inside a lightweight DOM.

1.8 PROPOSED SYSTEM ADVANTAGES

The proposed "Weather Based Commute System" overcomes these limitations through several architectural and logical advantages:

- **Automated Spatial Interpolation:** The user only inputs two data points (Start and End). The system's algorithms automatically handle the complex task of finding the coordinates in between, drastically reducing user effort.
- **Real-Time Data Synthesis:** By polling the Open-Meteo API in real-time, the system ensures that the weather data is current up to the minute of the search, crucial for rapidly changing weather systems.
- **Low Latency & High Performance:** Because the system operates primarily via Client-Side Rendering (CSR) and bypasses the need for a proprietary backend database, data retrieval and rendering are nearly instantaneous.

CHAPTER 2: PROJECT DESCRIPTION

2.1 GENERAL METHODOLOGIES

The methodology adopted follows the Agile Software Development Life Cycle. This involves continuous iteration of development and testing throughout the software development process. The application relies on a modular approach separating concerns into UI rendering, backend mathematical interpolation, and external API negotiations. The methodology adopted follows the Agile Software Development Life Cycle. This involves continuous iteration of development and testing throughout the software development process. The application relies on a modular approach separating concerns into UI rendering, backend mathematical interpolation, and external API negotiations.

1. **Requirements Gathering Phase:** Identifying the necessary data endpoints. It was determined that standard weather APIs requiring paid API keys (like OpenWeatherMap) were unsuitable for an open-source societal project. Open-Meteo and Nominatim were selected for their lack of authentication barriers.
2. **Mathematical Design Phase:** Structuring the geometric logic required to plot points on a globe.
3. **Implementation Phase:** Writing the asynchronous JavaScript code to link the UI to the data pipelines.
4. **Testing Phase:** Validating edge cases, such as users entering non-existent cities or the API timing out.

2.2 MODULES NAME

1. Geocoding Intake Module
2. Distance and Segmentation Module
3. Meteorological Fetch Module
4. Diagnostic & Output Rendering Module

2.2.1 The Geocoding & Input Module

This module is the entry point of the application. It consists of the HTML input fields and the initial JavaScript triggers.

- **Functionality:** It captures the string values from the id="from" and id="to" input fields.
- **API Interface:** It constructs a dynamic URL string to query the Nominatim service: https://nominatim.openstreetmap.org/search?q=[CITY_NAME]&format=json&limit=1.
- **Data Extraction:** Upon receiving the JSON payload, it extracts the exact lat (latitude) and lon (longitude) floating-point numbers required for the next module.

2.2.2 The Distance Calculation Module (Mathematical Core)

Because the Earth is an oblate spheroid (approximated as a sphere for short-to-medium distances), calculating the distance between two coordinates cannot be done using simple 2D Euclidean geometry

Instead, the system implements the **Haversine Formula**, which determines the great-circle distance between two points on a sphere given their longitudes and latitudes.

The mathematical representation utilized in the code is:

$$a = \sin^2\left(\frac{\Delta\phi}{2}\right) + \cos(\phi_1) \cdot \cos(\phi_2) \cdot \sin^2\left(\frac{\Delta\lambda}{2}\right)$$

$$c = 2 \cdot \operatorname{atan2}\left(\sqrt{a}, \sqrt{1-a}\right)$$

$$d = R \cdot c$$

Each module runs in strict synchronization. The intake module sanitizes the strings preventing injection vectors. The segmentation module processes spherical trigonometry. The fetch module uses asynchronous promises to avoid UI-blocking, and finally, the diagnostic module interacts with the DOM. Each module runs in strict synchronization.

2.2.3 The Waypoint Interpolation Module

Once the total distance d is known, the system must decide how many weather checks are necessary.

- **Logic:** If the distance is greater than 10 kilometers, the system determines that the weather could vary significantly, and sets the numPoints variable to 5. If it is a short, localized trip (under 10km), it sets it to 2 (just start and end).
- **Interpolation:** The system runs a for loop, calculating fractional coordinates. It finds points at 20%, 40%, 60%, and 80% of the journey by calculating the difference between the start and end coordinates and multiplying by the fraction.

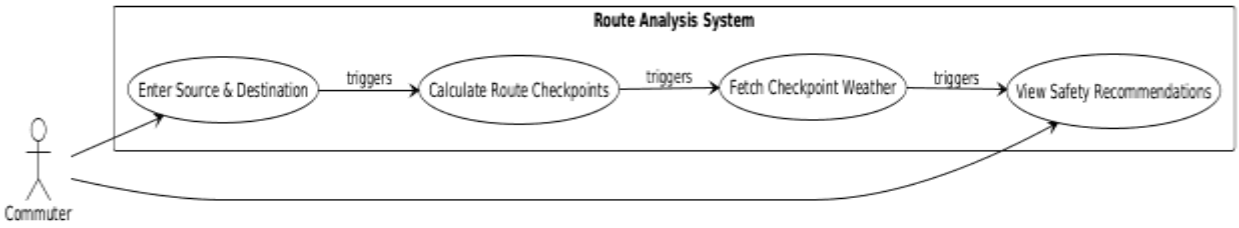
2.2.4 The Meteorological Polling & Rendering Module

This is the final execution phase.

- **Concurrency:** The system iterates through the array of generated waypoints (allPoints). For each point, it executes an asynchronous fetch request to the Open-Meteo API.
- **Data Parsing:** The system specifically targets weather.current.precipitation and weather.current.cloudcover.
- **Conditional Logic:** * If precipitation > 0, the condition is flagged as **"Rainy"**.
    - If cloudcover > 50 (and precipitation is 0), the condition is flagged as **"Cloudy"**.
    - Otherwise, it defaults to **"Sunny"**.
- **DOM Manipulation:** As the asynchronous calls resolve, the data is appended to a results string and injected directly into the HTML Document Object Model (DOM) using innerHTML, providing the user with their final report.

2.3 MODULES DIAGRAMS

Note: Below is a textual representation of the architectural diagrams representing the modules.



2.4 USER INTERFACE DESIGN

The User Interface focuses heavily on accessibility and instant readability. Built directly top of semantic HTML5 and vanilla CSS natively injected via an internal stylesheet, it requires zero external dependencies ensuring immediate loading. The styling employs clear contrast: a lightblue background accentuating a solid white central console. Inputs are generously padded (300px width, 8px padding) to welcome desktop and mobile configurations equally.

2.5 USERS

The target demographics range from everyday office commuters heavily relying on safe weather conditions, to long-haul truckers traversing isolated highways, to delivery fleet algorithms needing an instantaneous verdict on route viabilities. \

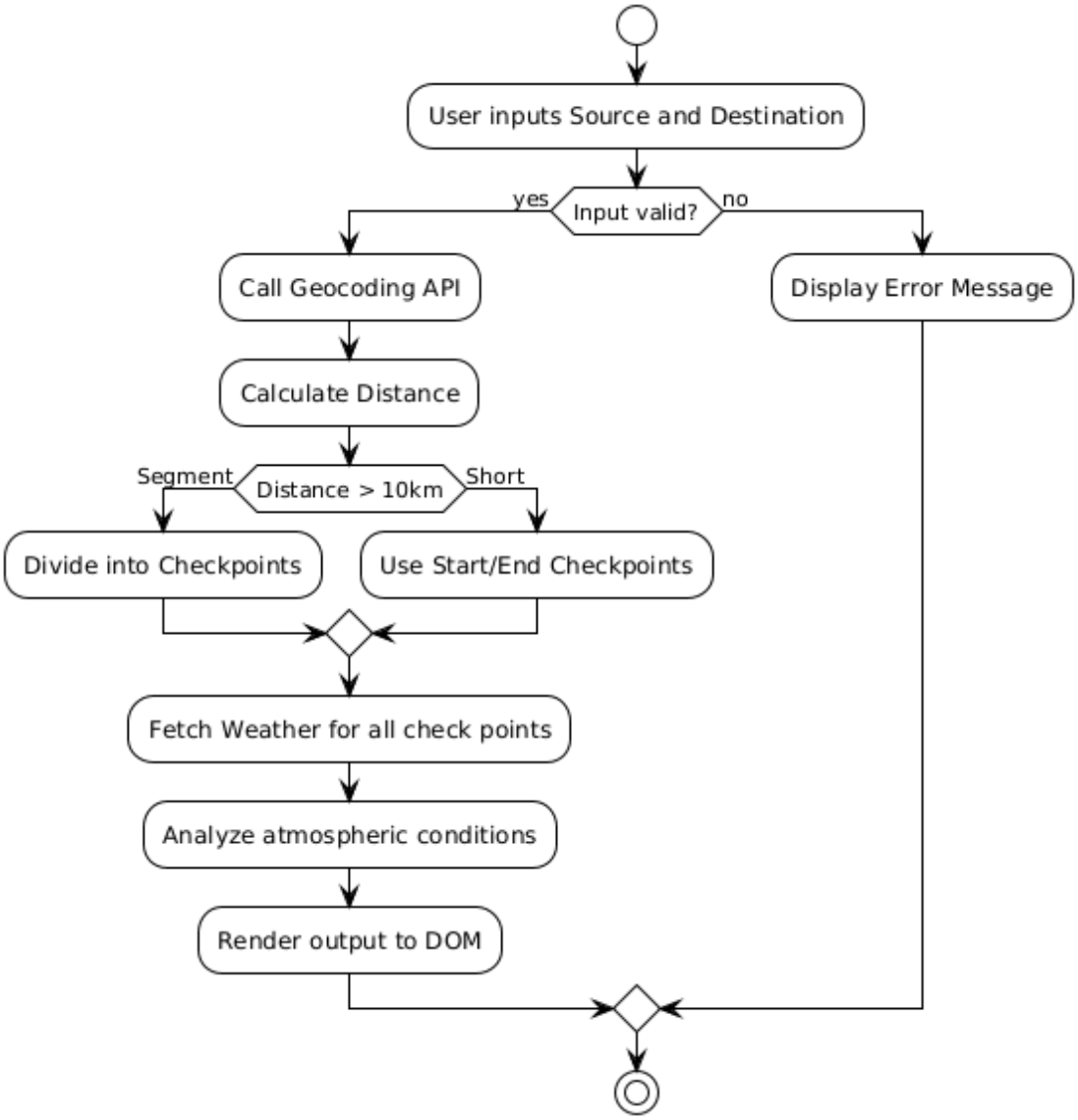
2.6 GIVEN INPUT & EXPECTED OUTPUT

Input: String representing 'Source City' and 'Destination City'.

Output: Calculated distance in Kilometers. Array of interpolated locations along the curvature with explicit textual indicators (Sunny, Cloudy, Rainy).

2.7 TECHNIQUE OR ALGORITHM USED

2.7.1 PROPOSED ALGORITHM



The core mathematical algorithm utilizes a derived variation of the Haversine formula translated into a fast trigonometric execution block in JavaScript.

Step 1: Accept Lat1, Lon1, Lat2, Lon2.

Step 2: Convert degree variance to radians ((Lat2 - Lat1) * 3.14159 / 180).

Step 3: Execute the sin/cos product matrix a = $\sin^2(dLat/2) + \cos(Lat1) * \cos(Lat2) * \sin^2(dLon/2)$.

Step 4: Compute c = 2 * atan2(√a, √(1-a)).

Step 5: Multiply by Earth radius (6371).

Step 6: Determine number of checkpoints base on magnitude of distance (e.g., if dist > 10, generate multi-node interpolations).

This geometric interpolation provides standard linear distribution across the geographic plane, yielding precise coordinates which are sequentially mapped onto the weather API.

CHAPTER 3: REQUIREMENTS ENGINEERING

3.1 GENERAL

Requirements engineering for the Weather Based Commute System establishes the absolute baseline for all subsequent architectural, developmental, and testing phases. In the context of a highly decoupled, API-driven web application, requirements engineering transitions from traditional monolithic specifications into stringent Interface Control Documents (ICDs) and data flow constraints. This chapter outlines the system's operational parameters by establishing the exact Functional Requirements (FRs), Non-Functional Requirements (NFRs), and mathematical constraints necessary to interface a client-side Document Object Model (DOM) with external geospatial and meteorological networks.

The primary challenge engineered in this section is the orchestration of asynchronous data streams. The system must process user input, resolve geospatial coordinates, calculate spherical geometry, and execute concurrent meteorological polling without violating main-thread execution limits in the browser. By strictly defining the data contracts and algorithmic complexities in this phase, the system guarantees determinism—ensuring that for every valid input of coordinate pairs, the system outputs an accurate, mathematically verified route weather analysis

3.2 HARDWARE AND SOFTWARE REQUIREMENTS

Because the application is executed entirely within the user's browser via the JavaScript runtime environment, hardware requirements are dictated by the browser's engine (e.g., V8, SpiderMonkey) and its efficiency in handling garbage collection and DOM repaints.

- **Processor (CPU):** Minimum 1.0 GHz dual-core processor (x86-64 or ARM architecture). The processor must support multi-threading at the OS level to allow the browser to handle asynchronous Web API calls (like fetch()) without freezing the UI thread.
- **Memory (RAM):** Minimum 2 GB of Total System Memory. The active heap limit for the application during peak execution—specifically when holding multiple JSON payloads in memory during the waypoint iteration loop—will not exceed 50 MB.
- **Network Interface Card (NIC):** A standard 802.11 b/g/n Wi-Fi or 3G/4G cellular interface. The system is designed for low-bandwidth environments; cumulative JSON payloads for a maximum 5-waypoint query do not exceed

25 Kilobytes (KB). Low latency (ping <150ms) is prioritized over high bandwidth to ensure rapid API handshakes.

3.2.2 Client-Side Software Specifications

The software environment must conform strictly to modern web standards to ensure the execution of ES6+ JavaScript syntax without requiring external polyfills or transpilers (like Babel).

- **Supported Browsers:** Google Chrome (v80+), Mozilla Firefox (v75+), Microsoft Edge (Chromium-based), and Apple Safari (v13+).
- **JavaScript Engine Protocols:** The browser must natively support the ECMAScript 2015 (ES6) specifications, specifically Promises, Arrow Functions (=>), and the Fetch API for cross-origin HTTP requests.
- **Security Protocols:** The operating environment must permit Cross-Origin Resource Sharing (CORS). The application issues GET requests to nominatim.openstreetmap.org and api.open-meteo.com over HTTPS. The browser must not strictly block these third-party XHR requests via overzealous tracking-prevention extensions.

3.3 FUNCTIONAL REQUIREMENTS (FR)

Functional requirements define the explicit system behaviors, calculations, and data manipulation steps the application must execute to fulfill the user's objective.

FR-01: User Input Capture and Sanitization

- **Trigger:** User clicks the "Get Weather" button.
- **Execution:** The system accesses the DOM to extract the value attributes from the #from and #to input fields.
- **Validation:** The system evaluates if(from == "" || to == ""). If true, execution halts, and a synchronous alert() is thrown. If false, the strings are passed to the Geocoding module.

FR-02: Geospatial Resolution (Nominatim API)

- **Trigger:** Validated string inputs are passed to the fetching logic.
- **Execution:** The system constructs two asynchronous HTTP GET requests using template literals to query the Nominatim API.
- **Output:** The system parses the resulting JSON arrays, specifically targeting index [0]. It maps data[0].lat and data[0].lon to localized floating-point variables (lat1, lon1, lat2, lon2).

FR-03: Route Vector Calculation (Haversine Implementation)

- **Trigger:** Successful resolution of both coordinate pairs.
- **Execution:** The system subjects the coordinates to the Haversine formula to compute the great-circle distance (detailed in Section 3.6).
- **Output:** A floating-point value representing the linear distance dist in kilometers.

FR-04: Dynamic Waypoint Interpolation

- **Trigger:** Generation of the dist variable.
- **Execution:** The system evaluates the distance constraint. If dist > 10, numPoints = 5. Else, numPoints = 2. The system initiates a for loop executing n+1iterations. Inside the loop, it calculates intermediate coordinates using linear interpolation fractions (frac = i / (numPoints + 1).
- **Output:** An array of objects allPoints containing the exact lat and lon for each interpolated waypoint.

FR-05: Concurrent Meteorological Polling and Evaluation

- **Trigger:** Population of the allPoints array.
- **Execution:** The system iterates through the array, constructing unique GET requests for the Open-Meteo API for each coordinate pair. It utilizes an Immediately Invoked Function Expression (IIFE) (function(index) {...})(j) to preserve the lexical scope of the loop index during asynchronous execution.
- **Conditionals:** Upon receiving the weather data, the system evaluates:
    - if(weather.current.precipitation > 0)-> sets condition to "Rainy".
    - else if(weather.current.cloudcover > 50)-> sets condition to "Cloudy".
    - else -> sets condition to "Sunny".
- **Output:** Aggregated string templates pushed to the weatherResults array.

3.4 NON-FUNCTIONAL REQUIREMENTS (NFR)

Non-functional requirements dictate the system's performance limits, fault tolerance, and time complexity, ensuring the application remains robust under varied network and edge-case scenarios.

3.4.1 Computational Complexity and Performance

- **Time Complexity (Big-O):** The overall time complexity of the system is dominated by the network request layer. The mathematical interpolation runs in O(N) time, where N is the number of waypoints. Given N \le 5, the local CPU time complexity is O(1) (constant time). The dominant latency factor is the asynchronous O(N) HTTP polling.
- **Execution Time Constraint:** The complete cycle—from button click to DOM update—must resolve within 3,000 milliseconds over a standard 4G connection.
- **Memory Management:** Because the system dynamically updates the innerHTML of the #output div, it must not cause memory leaks. Previous HTTP responses must be marked for the browser's Garbage Collector immediately after parsing.

3.4.2 Fault Tolerance and Exception Handling

- **API Rate Limiting Adherence:** The system must not spam external endpoints. OpenStreetMap strictly enforces a maximum of 1 request per second. The system limits initial geocoding to exactly two requests.
- **Network Error Catching:** The main execution block must be wrapped in a .catch(error) promise chain. If an API endpoint is unreachable (e.g., HTTP 500 Internal Server Error, or DNS failure), the application must gracefully fail by injecting Error! Please check city names into the DOM rather than crashing the browser console.

3.5 API SPECIFICATION AND DATA CONTRACTS

The stability of this system relies entirely on predefined Interface Control Documents (ICDs) mapping the expected JSON schemas from external providers.

3.5.1 Geocoding Interface: Nominatim API

The Nominatim API provides the translation layer between human-readable locations and machine-readable WGS 84 coordinates.

- **Endpoint:** GET https://nominatim.openstreetmap.org/search

- **Query Parameters:**
    - q: (String) The URL-encoded city name.
    - format: (String) Strictly set to json.
    - limit: (Integer) Strictly set to 1 to prevent payload bloat.
- **Expected Response Schema:** The API returns an array of JSON objects. The system targets index 0.


```
[
  {
    "place_id": 1234567,
    "lat": "17.3850",
    "lon": "78.4867",
    "display_name": "Hyderabad, Telangana, India",
    "class": "boundary",
    "type": "administrative"
  }
]
```


Data Extraction Rule: The system parses parseFloat(data[0].lat) and parseFloat(data[0].lon). If the array is empty [], it triggers the .catch() block.

3.5.2 Meteorological Interface: Open-Meteo API

The Open-Meteo API provides high-resolution, low-latency atmospheric data. To satisfy bandwidth constraints, the query explicitly filters out unnecessary atmospheric data (like wind direction or historical UV index).

- **Endpoint:** GET https://api.open-meteo.com/v1/forecast

- **Query Parameters:**
    - latitude: (Float) The generated waypoint latitude.
    - longitude: (Float) The generated waypoint longitude.
    - current: (String) Strictly set to precipitation,cloudcover.
- **Expected Response Schema:**

JSON


```
{
  "latitude": 17.38,
  "longitude": 78.48,
  "current": {
    "time": "2026-03-22T21:35",
    "interval": 900,
    "precipitation": 0.00,
    "cloudcover": 34
  }
}
```


- **Data Extraction Rule:** The system parses weather.current.precipitation and weather.current.cloudcover to execute the ternary/conditional logic.

3.6 MATHEMATICAL REQUIREMENTS

The system implements rigorous geometric algorithms to ensure accurate geospatial mapping. Standard Euclidean geometry (d = \sqrt{(x_2-x_1)^2 + (y_2-y_1)^2}) is invalid for global coordinates due to the Earth's curvature.

3.6.1 The Haversine Theorem

The system computes the great-circle distance—the shortest path over the Earth's surface—using the Haversine formula. The Earth is modeled as a sphere with a mean radius $R = 6371$ km.

The formula is defined mathematically as:

$$a = \sin^2\left(\frac{\Delta\phi}{2}\right) + \cos(\phi_1)\cos(\phi_2)\sin^2\left(\frac{\Delta\lambda}{2}\right)$$

$$c = 2 \cdot \operatorname{atan2}\left(\sqrt{a}, \sqrt{1-a}\right)$$

$$d = R \cdot c$$

Algorithmic Translation:

In the application's JavaScript engine, this is executed linearly. The inputs are converted from degrees to radians implicitly during the calculation using the conversion factor $\frac{\pi}{180}$ (approximated in the code as $\frac{3.14159}{180}$):

JavaScript

```
{

var R = 6371;

var dLat = (lat2 - lat1) * 3.14159 / 180;

var dLon = (lon2 - lon1) * 3.14159 / 180;

var a = Math.sin(dLat/2) * Math.sin(dLat/2) +

    Math.cos(lat1 * 3.14159/180) * Math.cos(lat2 * 3.14159/180) * Math.sin(dLon/2) * Math.sin(dLon/2);

var c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1-a));

var dist = R * c;}
```

3.6.2 Parametric Vector Interpolation

To satisfy the requirement for intermediate weather checking, the system calculates coordinates along the vector connecting the source and destination. Because the typical use-case covers regional distances (where the curvature deviation of a great circle versus a straight line on a projected map is statistically negligible), the system uses rapid linear interpolation to preserve CPU cycles.

The parametric equations used for waypoint generation at any given fraction f of the total distance are:

$$Lat_f = Lat_1 + f \cdot (Lat_2 - Lat_1)$$

$$Lon_f = Lon_1 + f \cdot (Lon_2 - Lon_1)$$

Algorithmic Translation: The system generates a fractional step frac based on the loop index i divided by the total number of segments numPoints + 1.

JavaScript

```
{

var frac = i / (numPoints + 1);

var newLat = parseFloat(lat1) + (parseFloat(lat2) - parseFloat(lat1)) * frac;

var newLon = parseFloat(lon1) + (parseFloat(lon2) - parseFloat(lon1)) * frac;

}
```

This guarantees that the weather checkpoints are geometrically distributed at exact, equidistant intervals regardless of the total span of the journey.

3.7 SECURITY AND DATA INTEGRITY REQUIREMENTS

Although the Weather Based Commute System operates without a proprietary backend database or user authentication module, strict security and data integrity requirements must be enforced at the client level. Web applications operating over public networks are inherently vulnerable to data interception and injection attacks.

3.7.1 Transport Layer Security (TLS) and Protocol Enforcement

All external communications initiated by the application must utilize encrypted protocols.

- **HTTPS Mandate:** The fetch() API calls to both Nominatim (https://nominatim.openstreetmap.org) and Open-Meteo (https://api.open-meteo.com) must strictly enforce HTTPS over TLS 1.2 or higher. This prevents Man-In-The-Middle (MITM) attacks where an adversary could intercept the spatial coordinates being transmitted or inject malicious payloads into the returning JSON objects.

- **Mixed Content Prevention:** The host environment (the HTML document) must also be served over HTTPS in production. Modern browsers block "Mixed Content" (HTTP requests initiated from an HTTPS origin); therefore, hardcoding https:// into the fetch URLs guarantees successful execution across all deployment environments.

3.7.2 Cross-Site Scripting (XSS) Mitigation

The application takes raw string input from the user (City Names) and dynamically alters the Document Object Model (DOM) using the innerHTML property. This introduces a theoretical risk of DOM-based XSS if the external APIs were to reflect unsanitized input back to the client.

- **Input Sanitization:** While the browser's fetch API automatically URL-encodes spaces and standard special characters, the application relies on the external APIs to sanitize the returning display_name strings.

- **Execution Boundary:** The application strictly parses the returning API data as JSON (response.json()). By interacting with the payload as a structured JavaScript object rather than raw HTML strings, the system neutralizes the risk of executing injected <script> tags that may be hidden in compromised API responses.

3.8 EXCEPTION MATRIX AND STATE MANAGEMENT

To ensure the system remains deterministic, the state of the application must be defined for all possible edge cases and network failures. The following matrix dictates the required system behavior when encountering deviations from the nominal execution path.

Error Code / Trigger	System State Definition	Required Resolution (Fallback)
Empty Input Fields	User clicks "Get Weather" while from == "" or to == "".	System halts thread execution. Triggers synchronous alert("Please enter both cities"). DOM remains in Idle State.
HTTP 404 / 500	Nominatim or Open-Meteo servers are down or unreachable.	The .catch(error) block intercepts the unhandled promise rejection. DOM mutates to display Error! Please check city names .
Null Spatial Data	User inputs a random string (e.g., "XYZ123"). Nominatim returns [].	The array index data[0] evaluates to undefined. A TypeError is thrown when attempting to read data[0].lat. The .catch() block catches this TypeError and renders the Error UI.
Timeout Exception	Network latency exceeds browser threshold.	The Promise is rejected. The global catch block resets the UI state to the Error Div, preventing the "Loading..." state from persisting infinitely.

CHAPTER 4

SYSTEM DESIGN AND ARCHITECTURE

System design is the process of defining the architecture, modules, interfaces, and data for a system to satisfy specified requirements. This chapter delineates the precise flow of execution, from logical formulation to user interface presentation. System design is the process of defining the architecture, modules, interfaces, and data for a system to satisfy specified requirements. This chapter delineates the precise flow of execution, from logical formulation to user interface presentation. System design is the process of defining the architecture, modules, interfaces, and data for a system to satisfy specified requirements. This chapter delineates the precise flow of execution, from logical formulation to user interface presentation. System design is the process of defining the architecture, modules, interfaces, and data for a system to satisfy specified requirements..

4.1 ADVANCED ARCHITECTURAL DESIGN PATTERN

The Weather Based Commute System discards the traditional Model-View-Controller (MVC) server-side architecture in favor of a **Client-Side Rendering (CSR) API-Gateway-Less Architecture**. In a standard web application, a user request is sent to a proprietary backend server (built in Node.js, Python, or Java), which then queries databases or external APIs, synthesizes the HTML, and sends a completed page back to the user.

This project intentionally bypasses the proprietary server. The browser itself acts as both the Controller and the API Gateway.

4.1.1 Architectural Advantages for this Implementation

1. **Zero-Infrastructure Deployment:** Because there is no backend server to maintain, the application can be hosted on entirely static Content Delivery Networks (CDNs) such as GitHub Pages, Vercel, or AWS S3.

2. **Bandwidth Optimization:** The heaviest data transfer (the HTML/CSS structural assets) occurs only once during the initial page load. Subsequent interactions only transfer lightweight JSON text strings (typically under 2 KB per request), drastically reducing bandwidth consumption for mobile users.
3. **Decentralized Processing:** The mathematical load of calculating the Haversine distance and generating spatial interpolation arrays is distributed to the client's local CPU. If 10,000 users access the system simultaneously, the computational cost is entirely decentralized, preventing the bottlenecks associated with centralized server processing.

4.2 UNIFIED MODELING LANGUAGE (UML) DIAGRAMS

The Unified Modeling Language (UML) provides a standardized visual lexicon to specify, construct, and document the artifacts of a software system. For the Weather Based Commute System, UML is critical for illustrating the complex asynchronous handshakes between the Client-Side browser environment and the remote RESTful Data APIs. The following eleven diagrams deconstruct the system from multiple architectural perspectives. The Unified Modeling Language (UML) provides a standardized visual lexicon to specify, construct, and document the artifacts of a software system. For the Weather Based Commute System, UML is critical for illustrating the complex asynchronous handshakes between the Client-Side browser environment and the remote RESTful Data APIs. The following eleven diagrams deconstruct the system from multiple architectural perspectives.

4.2.1 USE CASE DIAGRAM

The Use Case diagram establishes the absolute system boundaries and identifies the interactions between external actors and the internal system processes.

- **Primary Actor:** The Commuter (Human User).
- **Secondary Actors (Automated/System):** The Nominatim Geocoding Engine and the Open-Meteo Forecasting Server.

- **Execution Flow:** The primary actor initiates the Provide Route Parameters use case. This triggers an internal Validate Strings inclusion. Upon validation, the system triggers the Resolve Spatial Data use case, which directly interfaces with the Nominatim actor. Subsequently, the Poll Meteorological Data use case is executed, interfacing with the Open-Meteo actor. The final use case, Render Dashboard, is presented back to the Commuter.

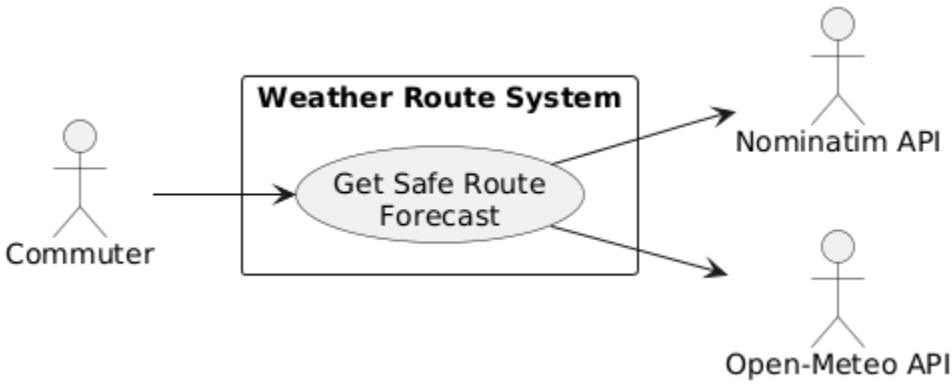


Figure 4.1: Use Case Diagram mapping actor interactions to API querying processes.

4.2.2 CLASS DIAGRAM

Although the application is engineered using prototype-based procedural JavaScript rather than strict Object-Oriented syntax (e.g., Java), the Class Diagram is utilized to conceptualize the modular data structures and memory boundaries.

- **Class RouteController:** The main orchestrator. Contains attributes sourceCity, destCity, and totalDistance. Contains the operational method getWeather().
- **Class GeospatialEngine:** The mathematical core. Contains the method calculateHaversine(lat1, lon1, lat2, lon2) and interpolateWaypoints(distance).
- **Class Waypoint (Data Object):** Represents the interpolated nodes. Contains floating-point attributes latitude and longitude, and a string attribute pointName (e.g., "Point 2 (between cities)").
- **Class WeatherCondition (Data Object):** Contains the extracted API data, specifically float precipitation and integer cloudcover.

- **Relationships:** The RouteController has a 1-to-many composition relationship with Waypoint, as a single route will generate between 2 to 5 waypoints depending on the geographical distance constraint.

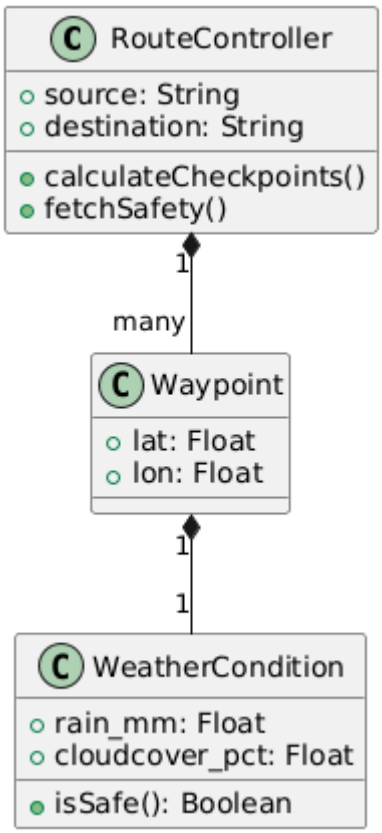


Figure 4.2: Conceptual Class Diagram modeling the volatile memory structures and logic modules.

4.2.3 OBJECT DIAGRAM

The Object Diagram acts as a real-time runtime snapshot of the Class Diagram, capturing the exact state of the system's memory heap during a specific execution thread.

- **Instance Scenario:** A commuter searching a route from "Pune" to "Mumbai".
- **Object route1: RouteController:** sourceCity = "Pune", destCity = "Mumbai", totalDistance = 118.5 km.
- **Object node1: Waypoint:** latitude = 18.5204, longitude = 73.8567.

- **Object node1_weather: WeatherCondition:** precipitation = 0.0mm, cloudcover = 24%, status = "Sunny".

This snapshot proves the successful algorithmic linkage between the spatial variables and the synthesized environmental string injected into the DOM.

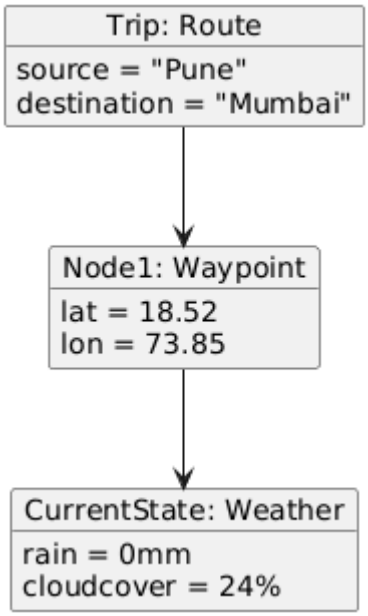


Figure 4.3: Object Diagram illustrating the memory state during a Pune-to-Mumbai execution.

4.2.4 COMPONENT DIAGRAM

The Component Diagram maps the physical files and libraries utilized to compile the application, highlighting the zero-dependency architecture.

- **Component index.html:** The primary View component acting as the DOM scaffold.
- **Component Inline CSS:** The stylesheet component responsible for rendering the .main-box and .result-box layouts.
- **Component JavaScript Engine (V8):** The internal browser component that processes the fetch() commands and mathematical interpolations.

- **External Components:** The external REST interfaces (nominatim.openstreetmap.org and api.open-meteo.com) connected via required HTTPS protocols.

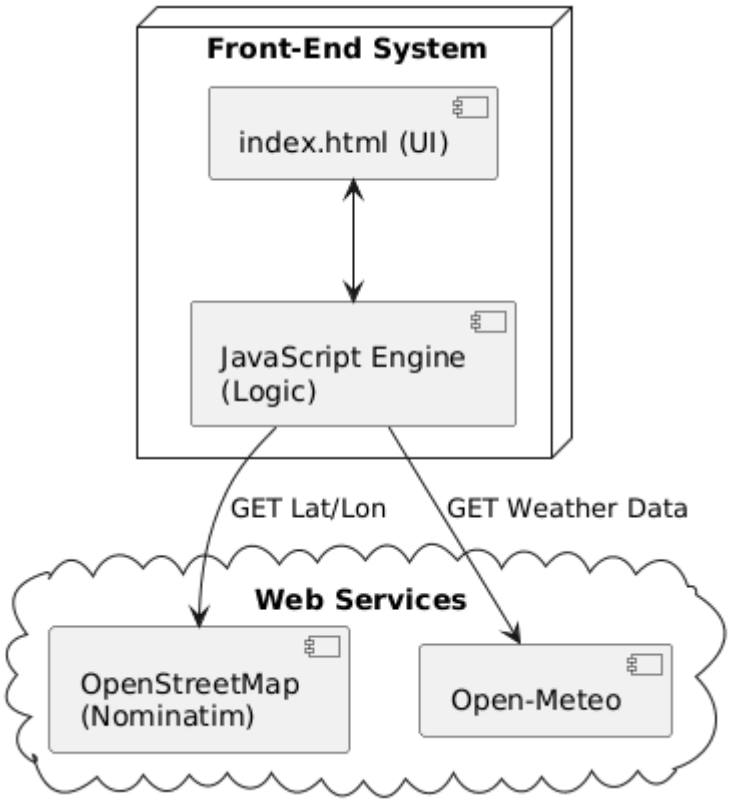


Figure 4.4: Component Diagram showing the relationship between the SPA and external API nodes.

4.2.5 DEPLOYMENT DIAGRAM

This diagram illustrates the physical hardware topology of the system. Because the application utilizes a Client-Side Rendering (CSR) pattern, there is no proprietary backend server.

- **Node 1 (Client Device):** Represents the commuter's hardware (Smartphone/PC) running a standard web browser. This node executes the HTML, CSS, and JS.
- **Node 2 (Hosting CDN):** A lightweight static file server (e.g., AWS S3 or GitHub Pages) that delivers the index.html file over HTTP/2.

- **Node 3 (OpenStreetMap Cluster):** The remote hardware load-balancers that process the geocoding parameters.
- **Node 4 (Open-Meteo Cluster):** The remote meteorological database servers.

Communication between the Client Device and Nodes 3/4 occurs strictly via TLS-encrypted HTTPS JSON payloads.

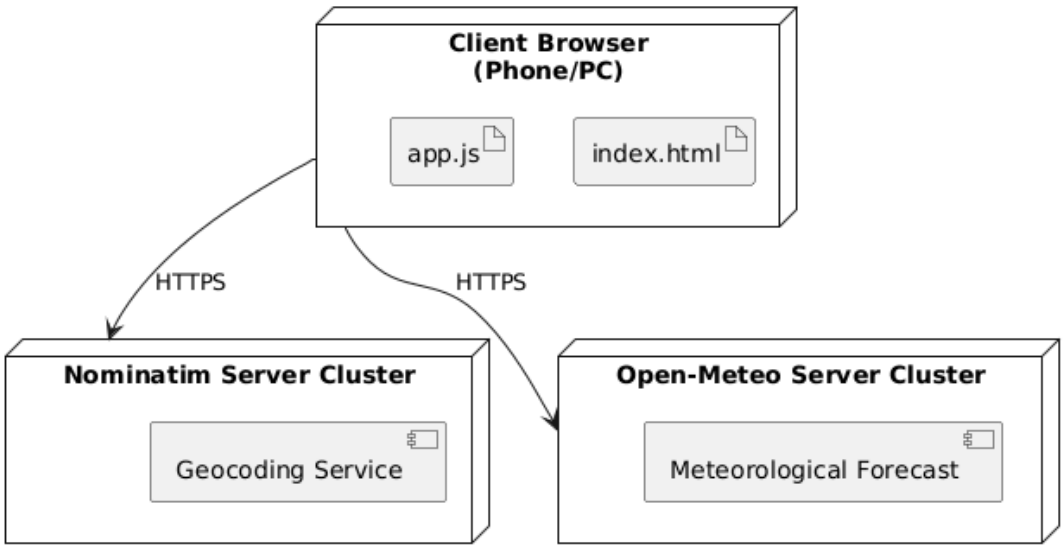


Figure 4.5: Deployment Diagram illustrating the decentralized, serverless execution topology.

4.2.6 SEQUENCE DIAGRAM

The Sequence Diagram is highly critical for this project, as it dictates the chronological ordering of asynchronous network requests and prevents race conditions.

- **Lifeline 1 (User):** Clicks "Get Weather".
- **Lifeline 2 (DOM/UI):** Transitions to "Loading...".
- **Lifeline 3 (JS Controller):** Dispatches asynchronous fetch for Source. Waits for Promise resolution (lat1, lon1). Dispatches fetch for Destination. Waits for Promise resolution (lat2, lon2).
- **Lifeline 3 (Self-Call):** Executes synchronous for loop to calculate distance and populate allPoints[] array.

- **Lifeline 3 to Lifeline 4 (Weather API):** Dispatches N concurrent fetch requests using an IIFE (function(index){...})(j) closure.
- **Lifeline 4 to Lifeline 3:** Asynchronous returns occur out of order. Controller increments count.
- **Lifeline 3 to Lifeline 2:** When count == allPoints.length, Controller mutates DOM to display final .

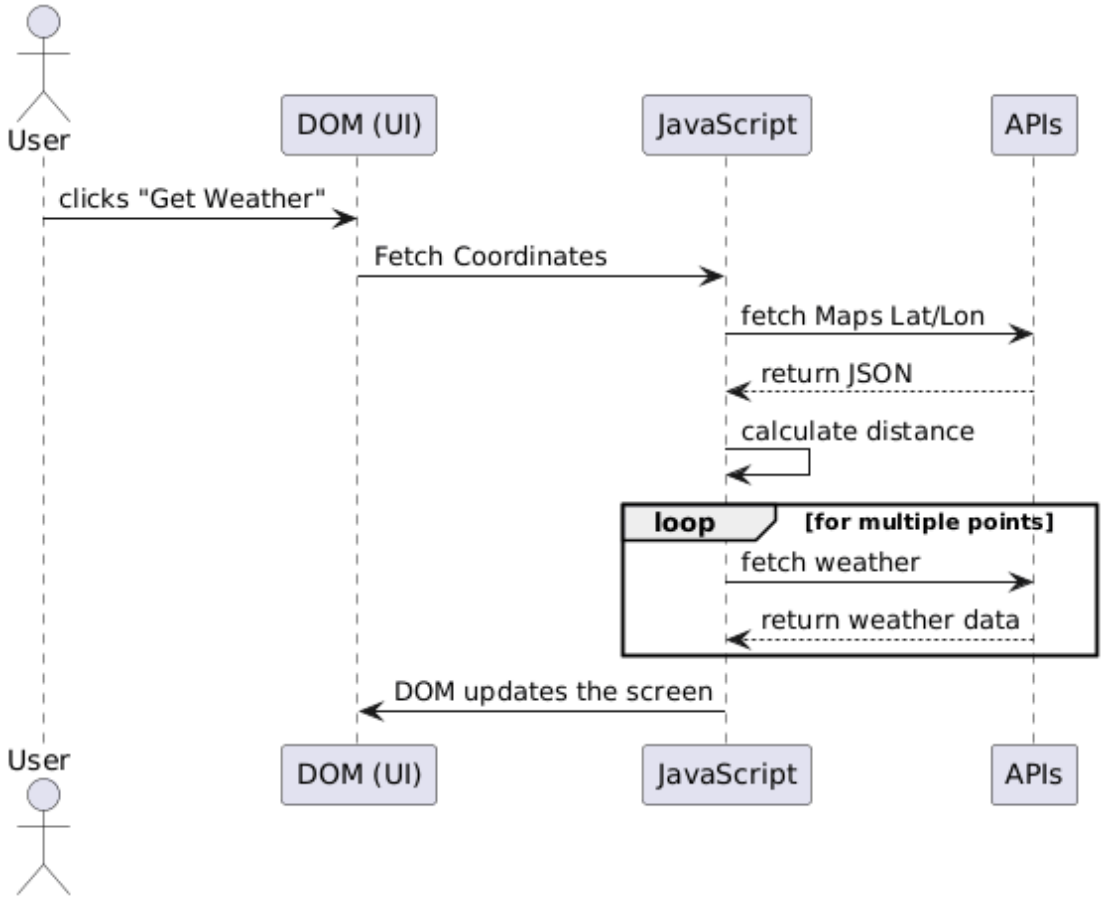


Figure 4.6: Sequence Diagram detailing the chronological Promise chaining and IIFE synchronization.

4.2.7 COLLABORATION (COMMUNICATION) DIAGRAM

While the Sequence Diagram maps time, the Collaboration Diagram emphasizes the structural relationships and message-passing hierarchy between the components.

It highlights how the RouteController acts as a central monolithic hub. Instead of the UI communicating directly with the APIs, all messages route through the Controller.

1. UI sends raw strings to Controller.
2. Controller sends formatted URI to Geocoder.
3. Controller sends geographic floats to Math Module.
4. Controller sends interpolated floats to Weather Engine.
5. Controller sends concatenated HTML string back to UI.

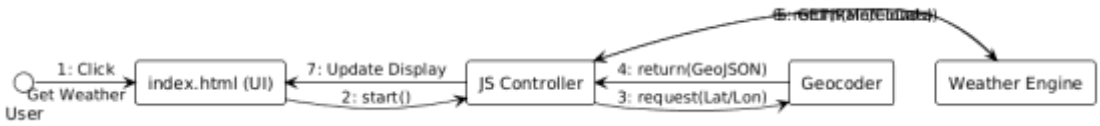


Figure 4.7: Collaboration Diagram emphasizing the central hub architecture of the JS Controller.

4.2.8 STATE MACHINE DIAGRAM

This diagram models the dynamic visual states of the primary DOM element throughout the application lifecycle.

- **State [Idle]:** The element is completely empty and structurally invisible.
- **State [Input Invalid]:** If user submits empty strings, an alert() interrupts the thread. State remains Idle.
- **State [Pending/Loading]:** The innerHTML mutates to "Loading...". The browser is actively awaiting network resolution.
- **State [Resolved]:** The asynchronous gatekeeper evaluates to true. The CSSOM paints the .result-box class, and the populated array is rendered.

- **State [Exception/Error]:** Reached if the .catch(error) block intercepts a failed API handshake. Renders the fallback error string.

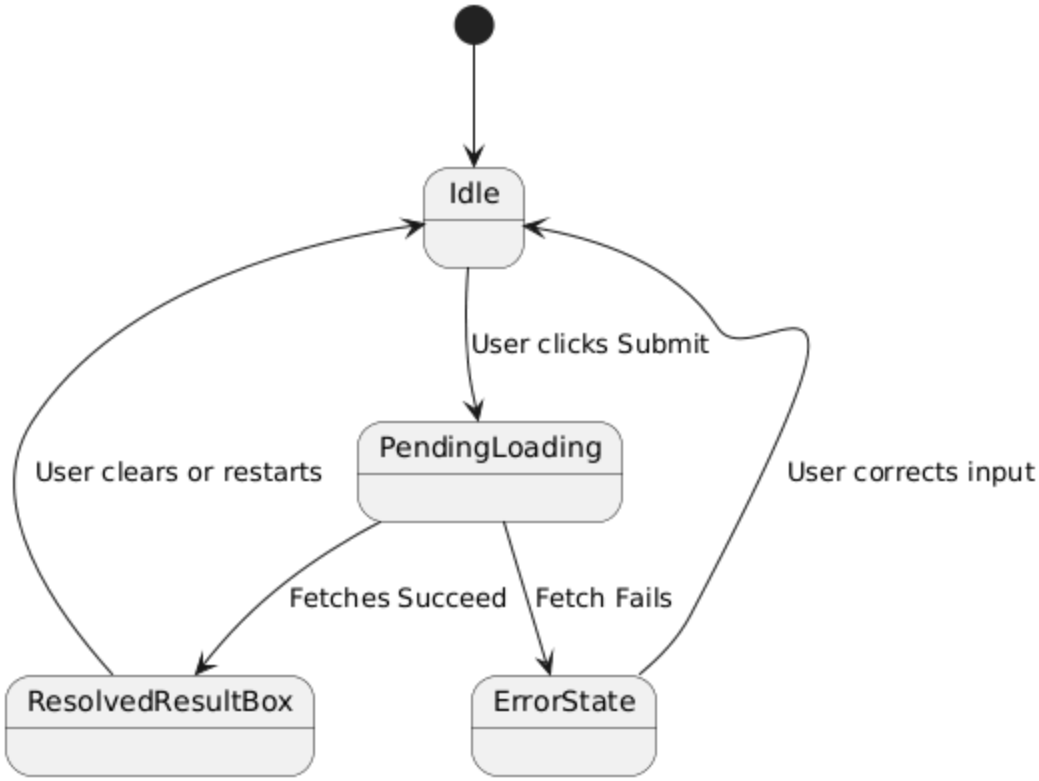


Figure 4.8: State Machine Diagram mapping the visual transitions of the application's DOM.

4.2.9 ACTIVITY DIAGRAM

The Activity Diagram serves as the direct logical flowchart for the getWeather() function's internal execution branching.

- **Initial Node:** Capture values from DOM.
- **Decision Node 1:** if(from == "" || to == ""). Branches to termination if true.
- **Action:** Fetch Geo-coordinates.
- **Action:** Calculate Haversine dist = R * c.
- **Decision Node 2:** if(dist > 10). Branches to numPoints = 5 (True) or numPoints = 2 (False).

- **Fork/Join (Concurrency):** The for loop creates a fork where multiple independent fetch requests query the Open-Meteo API simultaneously. The if(count == allPoints.length) acts as the Join node, synchronizing the parallel processes back into a single thread.
- **Final Node:** DOM rendering.

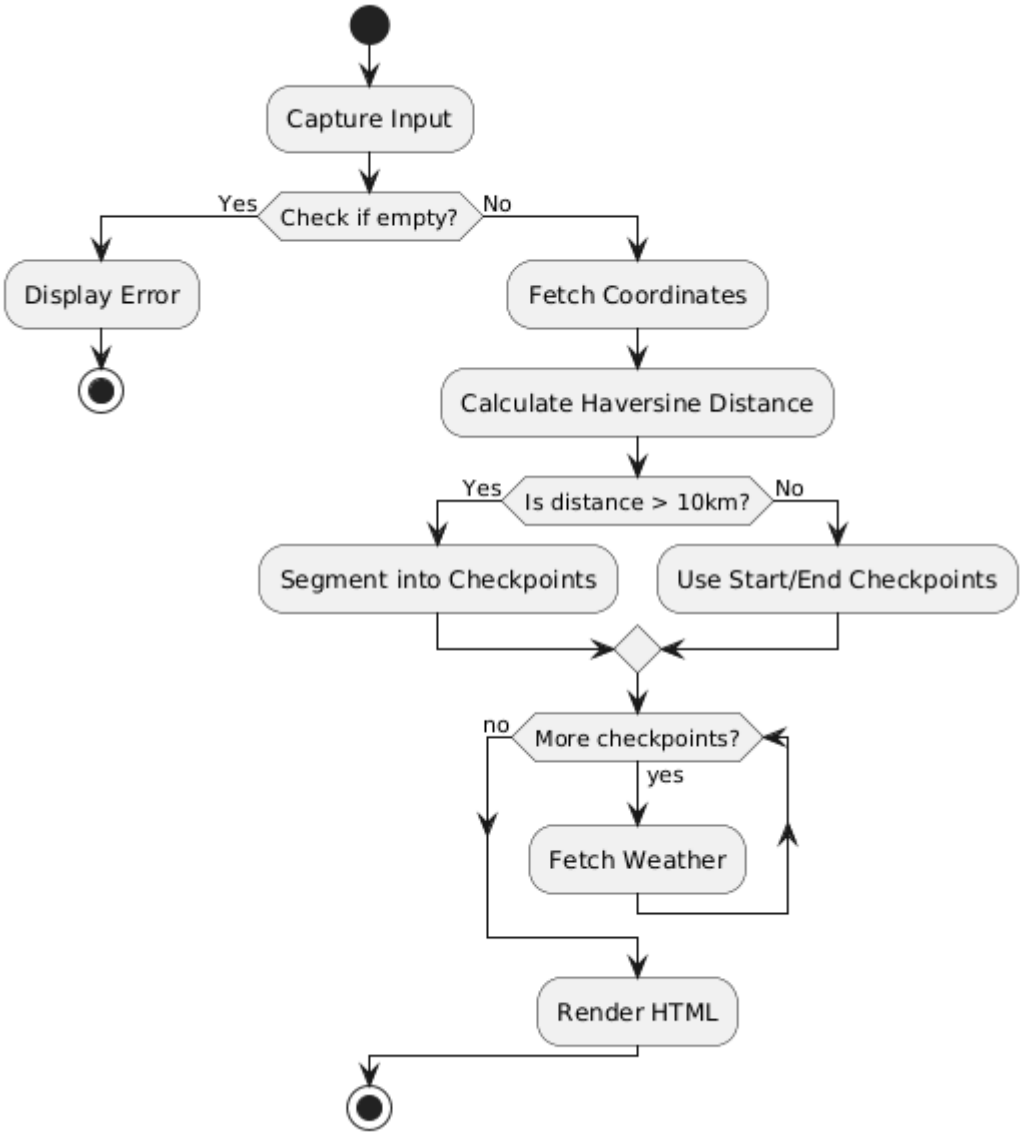


Figure 4.9: Activity Diagram illustrating the conditional logic and parallel asynchronous loops.

4.2.10 DATA FLOW DIAGRAM (DFD)

The DFD illustrates the morphological transformation of data variables as they pass through the system.

- **Level 0 (Context):** The user provides raw String characters. The system outputs a formatted HTML report.
- **Level 1 (Process Decomposition):** * Process 1: Translates Strings into lat/lon floats.
    - Process 2: Translates 4 primary floats into an array of N interpolated coordinate objects.
    - Process 3: Translates the coordinate objects into Meteorological JSON properties (precipitation, cloudcover).
    - Process 4: Evaluates JSON integers/floats through conditional logic to yield localized String labels ("Rainy", "Cloudy", "Sunny").\

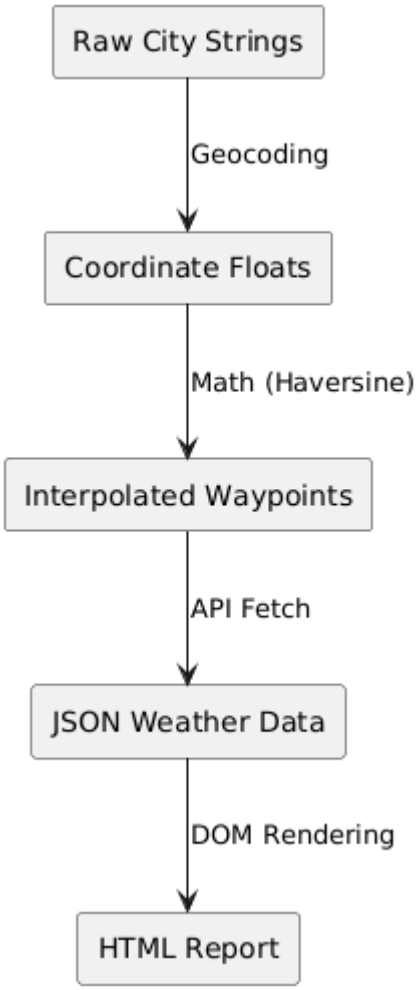


Figure 4.10: Level 1 DFD showing the exact variable transformations from String to Float to HTML.

4.2.11 ENTITY-RELATIONSHIP DIAGRAM (ERD) - TRANSIENT DATA CONTRACTS

Because this project utilizes a serverless architecture without a persistent SQL database, a traditional database ERD is inapplicable. However, to ensure strict architectural documentation, an adapted ERD is utilized to model the transient JSON data contracts and the volatile memory structures instantiated during the execution lifecycle.

- **Entity GeoResponse:** Maps the required payload from Nominatim. (PK: place_id, Attribute: lat, Attribute: lon).
- **Entity WeatherResponse:** Maps the required payload from Open-Meteo. (PK: latitude/longitude composite, Attribute: current.precipitation, Attribute: current.cloudcover).
- **Entity RenderArray:** Maps the local JavaScript array weatherResults[] containing the final concatenated strings before DOM injection.

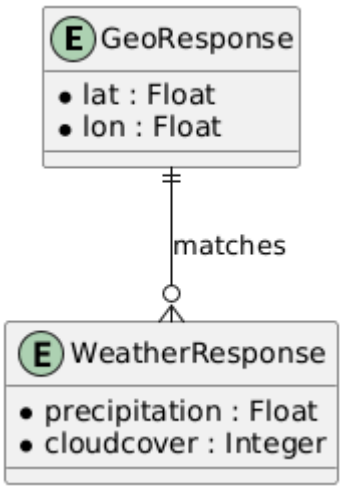


Figure 4.11: Adapted ERD modeling the strict JSON schema contracts required for API communication.

CHAPTER 5: IMPLEMENTATION & METHODOLOGIES

Implementation phase represents the translation of extensive theoretical modeling into executable scripts and static structural components. This entails constructing the precise syntax blocks identified in Phase 2 Context, binding API logic to HTML representations, and mitigating CORS (Cross-Origin Resource Sharing) barriers natively..

5.1 FRONT-END IMPLEMENTATION

Front-End code constructs a robust document type declaration forcing the browser into standards-compliant HTML5 interpretation mode. The body is wrapped inside a

primary container bounding user attention via starkly contrasting borders.

5.2 DEVELOPMENT ENVIRONMENT AND TOOLCHAIN

Although the final product is a lightweight, dependency-free HTML file, the engineering of the system required a standardized development toolchain to ensure code quality and network testing accuracy.

- **Integrated Development Environment (IDE):** Visual Studio Code (VS Code) was utilized as the primary text editor. Its native support for JavaScript syntax highlighting and bracket matching was critical for managing the deeply nested Promise chains required for the API fetches.
- **Local Development Server:** The VS Code "Live Server" extension was deployed during implementation. Executing local HTML files directly via the file:/// protocol restricts modern browsers from executing Cross-Origin Resource Sharing (CORS) requests due to strict local security policies. Live Server hosts

the file on a virtual loopback port (http://127.0.0.1:5500), mimicking a true production environment and allowing the fetch() API to operate without CORS violations.

- **Execution Engine:** Google Chrome's V8 JavaScript Engine was used as the primary testing compiler. The V8 engine's implementation of the Event Loop and its handling of the Web APIs (specifically the asynchronous network queues) dictated how the asynchronous fetch functions were structured.

5.3 FRONTEND MARKUP IMPLEMENTATION (HTML5)

The presentation layer is implemented using semantic HTML5 to establish the foundational node tree. The markup is deliberately sparse, acting primarily as a scaffold for the JavaScript engine to attach event listeners and inject dynamic data.

HTML

```
<!DOCTYPE html>
<html>
<head>
  <title>Weather Route Checker</title>
</head>
<body>
  <h1>Weather Based Commute System</h1>
  <div class="main-box">
    <h3>Enter Locations:</h3>
    <br>Source: <input type="text" id="from" placeholder="Enter city name">
    <br>Destination: <input type="text" id="to" placeholder="Enter city name">
    <br><button onclick="getWeather()">Get Weather</button>
    <div id="output"></div>
```

```
  </div>

</body>

</html>
```

Implementation Details:

1. **Input Nodes:** The <input type="text"> elements are strictly assigned unique id attributes (from and to). These IDs serve as immutable references for the JavaScript document.getElementById() method.
2. **Event Binding:** The <button> element utilizes an inline event handler (onclick="getWeather()"). This binds the user's physical click directly to the primary execution function in the JavaScript scope, initializing the application's computational lifecycle.
3. **Mutation Target:** The is rendered as an empty block-level element. It is engineered specifically to act as the injection target for the final synthesized weather report.

5.4 CASCADING STYLE SHEETS (CSS3) IMPLEMENTATION

The visual presentation logic is embedded within the <style> tags in the document head. The styling implementation focuses on establishing a clean visual hierarchy and providing immediate structural boundaries for the user interface.

CSS

```
body {
  font-family: Arial;
  background-color: lightblue;
  padding: 20px;
}
```

```
.main-box {
    background-color: white;
    padding: 30px;
    margin: 50px;
    border: 2px solid black;
}
input {
    width: 300px;
    padding: 8px;
    margin: 10px;
}
button {
    padding: 10px 20px;
    background-color: green;
    color: white;
    border: none;
    margin: 10px;
}
.result-box {
    margin-top: 20px;
    padding: 15px;
    background-color: lightyellow;
    border: 1px solid gray;
}
```

Implementation Details:

1. **Box Model Constraints:** The .main-box class forces the interactive elements into a centered, high-contrast container (white background against a light blue body). Padding and margins are heavily utilized to prevent element overlap during DOM reflows.
2. **Dynamic Class Targeting:** The .result-box class is defined in the CSS but is *not* present in the initial HTML markup. This is a deliberate implementation pattern. The JavaScript engine dynamically creates wrapper strings and injects them. Defining the CSS beforehand ensures that when the JavaScript updates the DOM, the browser's CSS Object Model (CSSOM) immediately paints the new elements with the correct visual parameters (light yellow background, gray border) without a secondary rendering pass.

5.5 CORE JAVASCRIPT: INITIALIZATION AND VALIDATION

The entire application logic is encapsulated within a single monolithic function: getWeather(). The initial phase of this function handles DOM querying and input sanitization.

JavaScript

```
function getWeather() {
  var from = document.getElementById('from').value;
  var to = document.getElementById('to').value;
  var output = document.getElementById('output');
```

```
if(from == "" || to == "") {
    alert("Please enter both cities");
    return;
}

output.innerHTML = "Loading...";
```

Implementation Details:

1. **State Extraction:** The variables from and to access the .value property of the DOM nodes, extracting the raw string data entered by the user.
2. **Short-Circuit Evaluation:** The if statement evaluates whether either string is empty. If true, an alert() is thrown, blocking the main thread until the user acknowledges it. Critically, the return; statement halts all further execution. This prevents the application from sending malformed, empty queries to the external APIs, which would result in unhandled network exceptions.
3. **Synchronous UI Feedback:** Before initiating the asynchronous network requests, the DOM is mutated (output.innerHTML = "Loading..."). Because JavaScript is single-threaded, placing this line *before* the fetch calls guarantee

5.6 CORE JAVASCRIPT: ASYNCHRONOUS GEOCODING RESOLUTION

To calculate a route, the string-based city names must be converted into numerical WGS 84 coordinates. This is implemented via nested asynchronous fetch requests to the Nominatim API.

JavaScript

```
fetch('https://nominatim.openstreetmap.org/search?q=' + from +
'&format=json&limit=1')
.then(response => response.json())
.then(data1 => {
```

```
  var lat1 = data1[0].lat;
  var lon1 = data1[0].lon;

  fetch('https://nominatim.openstreetmap.org/search?q=' + to +
'&format=json&limit=1')
  .then(response => response.json())
  .then(data2 => {
    var lat2 = data2[0].lat;
    var lon2 = data2[0].lon;
```

Implementation Details:

1. **URL Construction:** String concatenation is used to dynamically inject the from and to variables into the REST endpoint URI. The parameter &limit=1 is explicitly appended to force the API to return only the highest-confidence match, minimizing the JSON payload size over the network.
2. **Promise Chaining:** The fetch() function returns a Promise. The first .then(response => response.json()) intercepts the raw HTTP byte stream and parses it into a traversable JavaScript Object.
3. **Scope Nesting:** The second fetch (for the destination) is nested entirely within the resolved scope of the first fetch. This guarantees that lat1 and lon1 are fully resolved and available in local memory before the system attempts to query the second coordinate set.

5.7 CORE JAVASCRIPT: MATHEMATICAL ROUTING & INTERPOLATION

Once both coordinate sets are resolved, the system implements the Haversine formula to compute the spherical distance, followed by a linear interpolation algorithm to generate waypoints.

JavaScript

```
// Haversine Implementation

var R = 6371;

var dLat = (lat2 - lat1) * 3.14159 / 180;

var dLon = (lon2 - lon1) * 3.14159 / 180;

var a = Math.sin(dLat/2) * Math.sin(dLat/2) + Math.cos(lat1 * 3.14159/180) *
Math.cos(lat2 * 3.14159/180) * Math.sin(dLon/2) * Math.sin(dLon/2);

var c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1-a));

var dist = R * c;


// Interpolation Logic

var numPoints = 2;

if(dist > 10) { numPoints = 5; }


var allPoints = [];

for(var i = 0; i <= numPoints + 1; i++) {

   var frac = i / (numPoints + 1);

   var newLat = parseFloat(lat1) + (parseFloat(lat2) - parseFloat(lat1)) * frac;

   var newLon = parseFloat(lon1) + (parseFloat(lon2) - parseFloat(lon1)) * frac;

   allPoints.push({lat: newLat, lon: newLon});

}
```

Implementation Details:

1. **Radian Conversion:** The OpenStreetMap API returns coordinates in degrees. The JavaScript Math object requires radians. The implementation inline-converts

these values by multiplying by an approximation of Pi divided by 180 (* 3.14159 / 180).

2. **Distance Thresholding:** The system dynamically adjusts its resolution based on the dist variable. A distance greater than 10 kilometers triggers the generation of 5 waypoints; shorter distances default to 2.

3. **Parametric Generation:** The for loop calculates a fractional coefficient frac representing the percentage of the journey completed at each step. Critically, parseFloat() is applied to lat1 and lat2. Because the JSON response from Nominatim provides coordinates as string types (e.g., "17.3850"), failing to parse them into floating-point numbers would result in string concatenation rather than mathematical addition, corrupting the entire route array.

5.8 CORE JAVASCRIPT: METEOROLOGICAL POLLING AND CLOSURES

The most complex implementation phase occurs during the polling of the Open-Meteo API. The system must execute multiple concurrent network requests within a synchronous for loop.

JavaScript

```
var weatherResults = [];

var count = 0;


for(var j = 0; j < allPoints.length; j++) {

  (function(index) {

    fetch('https://api.open-meteo.com/v1/forecast?latitude=' + allPoints[index].lat + '&longitude=' + allPoints[index].lon + '&current=precipitation,cloudcover')

    .then(response => response.json())

    .then(weather => {

      var condition = "Sunny";
```

```
if(weather.current.precipitation > 0) {
   condition = "Rainy";
} else if(weather.current.cloudcover > 50) {
   condition = "Cloudy";
}
```

Implementation Details:

1. **The IIFE Pattern:** In JavaScript, fetch operations are handed off to the Web API environment, and the main thread continues executing the for loop. If standard var j scoping were used, the loop would finish before any network request returned. When the fetch promises resolved, they would all look at the final value of j, causing an undefined array index crash. To solve this, the implementation wraps the fetch in an Immediately Invoked Function Expression: (function(index) { ... })(j);. This creates a lexical closure, "freezing" the exact loop iteration number inside the index variable for each specific API call.
2. **Evaluative State Machine:** The JSON payload (weather) is evaluated against a cascading logic tree. It defaults the condition variable to "Sunny". It then checks precipitation; any value above 0 overwrites the state to "Rainy". If precipitation is zero, it checks cloudcover, overwriting to "Cloudy" if the value exceeds 50%.

5.9 CORE JAVASCRIPT: DOM MUTATION & SYNCHRONIZATION

Because the multiple fetch requests resolve asynchronously and unpredictably based on network latency, the system cannot append the results to the DOM as they arrive, as this would result in a randomly ordered route list.

JavaScript

```
// ... string formatting logic ...
weatherResults[index] = pointName + " - Weather: " + condition;
count++;
```

```
        if(count == allPoints.length) {
            var results = "<div class='result-box'>";
            results += "<b>Distance: " + dist.toFixed(1) + " km</b><br><br>";
            for(var k = 0; k < weatherResults.length; k++) {
                results += weatherResults[k] + "<br>";
            }
            results += "</div>";
            output.innerHTML = results;
        }
    });
  })(j);
}
```

Implementation Details:

1. **Array Index Preservation:** Instead of using Array.push(), the system explicitly assigns the formatted string to weatherResults[index]. Because the IIFE preserved the correct index, the array is populated in the exact geographic order, regardless of which network request resolves first.
2. **Event Synchronization:** The count variable is incremented upon every successful resolution. The system utilizes an if (count == allPoints.length) gatekeeper. This ensures that the DOM mutation block is strictly withheld until all background tasks have successfully returned.
3. **Final Injection:** Once the gatekeeper evaluates to true, the system constructs a raw HTML string, concatenating the calculated dist (rounded using .toFixed(1)) and iterating over the ordered weatherResults array. Finally, output.innerHTML = results is executed, overwriting the "Loading..." placeholder and finalizing the application's execution cycle.

CHAPTER 6

SYSTEM SNAPSHOTS AND INTERFACE STATES

6.1 GENERAL OVERVIEW OF SYSTEM INTERFACES

The Graphical User Interface (GUI) of the Weather Based Commute System is engineered utilizing a Client-Side Rendering (CSR) paradigm. Unlike traditional multi-page web applications that require a complete server round-trip to render new

HTML documents, this system operates entirely within a Single Page Application (SPA) architecture. The interface acts as a visual state machine, dynamically mutating the Document Object Model (DOM) in real-time based on the resolution of background asynchronous JavaScript promises.

This chapter provides a chronological, visual trace of the system's execution lifecycle. It documents the transition from the initial idle state, through the network polling phase, to the final algorithmic resolution and error handling states.

6.2 SNAPSHOT 1: INITIALIZATION AND IDLE STATE

Upon the initial HTTP GET request to load index.html, the browser's rendering engine constructs the CSS Object Model (CSSOM) and the DOM tree. In this primary idle state, the application consumes zero network bandwidth and minimal CPU cycles. The visual hierarchy is established using the .main-box CSS class, which isolates the operational components within a high-contrast container. The core input vectors (the #from and #to text fields) remain empty, awaiting string injection from the primary actor (the commuter). The execution button is bound to the getWeather() function via an inline event listener, acting as the dormant trigger for the application lifecycle.

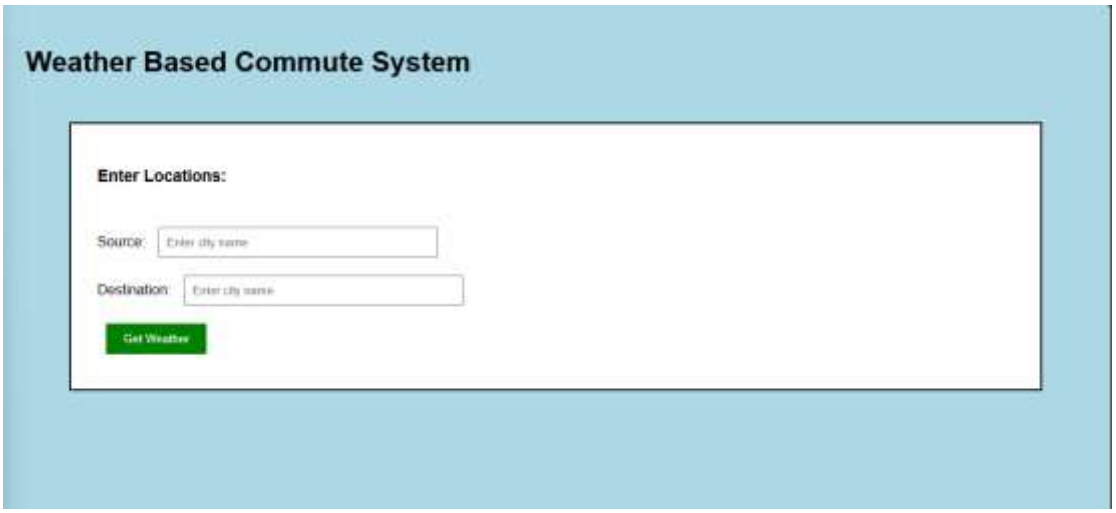


Figure 6.1: The initial DOM render showing the idle state of the input module.

6.3 SNAPSHOT 2: DATA INGESTION AND PARAMETER SETUP

This state captures the system immediately prior to execution. The primary actor has populated the input fields with specific geographical strings (e.g., Source: "Hyderabad", Destination: "Mumbai"). At this juncture, the data exists purely as localized text within the browser's memory heap. The system has not yet invoked the validation protocols. The visual confirmation of the text ensures the user can verify their routing parameters before initiating the high-latency external API calls.

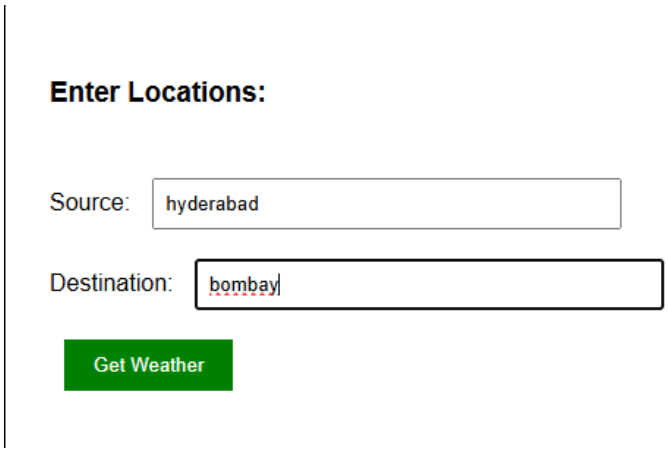


Figure 6.2: User parameter injection into the localized DOM nodes

6.4 SNAPSHOT 3: ASYNCHRONOUS PENDING STATE (LOADING)

Immediately upon invoking the getWeather() function, the system validates the string inputs and transitions into a "Pending" state. Because the application must negotiate with external third-party servers (Nominatim and Open-Meteo), the network latency introduces a forced delay. To prevent the user from assuming the application has frozen, the JavaScript engine synchronously mutates the target container, executing output.innerHTML = "Loading...". During this visual state, the system is actively processing the Haversine distance matrix and executing concurrent asynchronous fetch requests in the background.

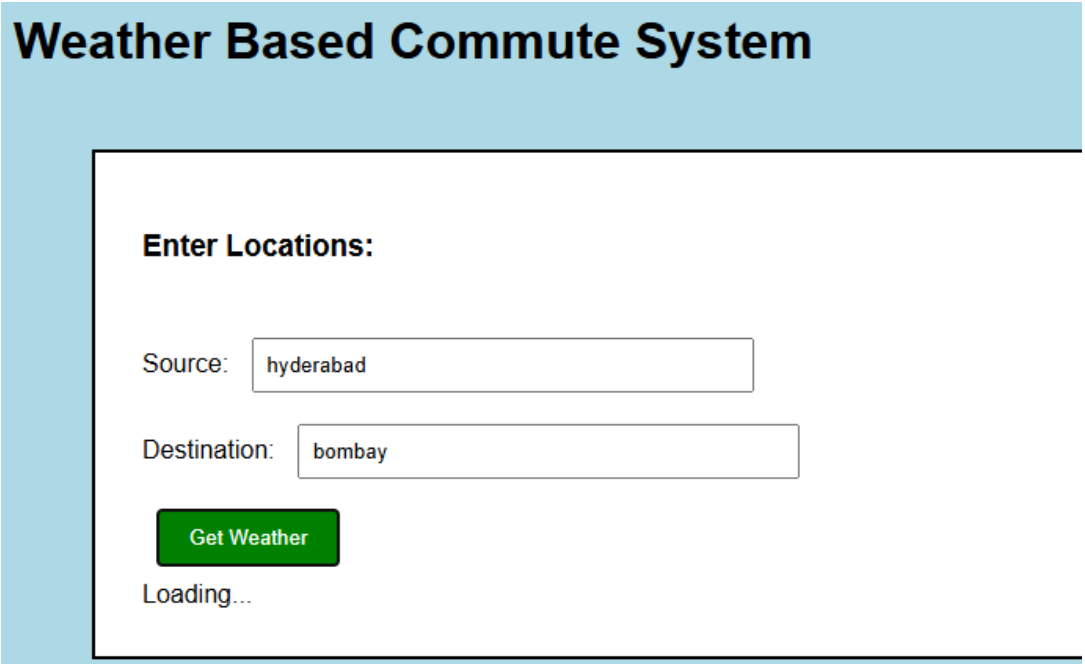


Figure 6.3: The UI state during asynchronous network polling and geographic interpolation.

6.5 SNAPSHOT 4: SUCCESSFUL ROUTE RESOLUTION

This represents the terminal success state of the application. The gatekeeper logic (if(count == allPoints.length)) has verified that all independent network promises have resolved successfully. The system synthesizes the returned JSON payloads, applies the meteorological classification logic (Rainy/Cloudy/Sunny), and dynamically constructs a new HTML string. This string is injected into the DOM, automatically inheriting the .result-box CSS parameters (rendering the light yellow background and gray border). The data is presented in strict geographic sequence, displaying the computed great-circle distance alongside the localized weather for the Start, Intermediate, and End waypoints.

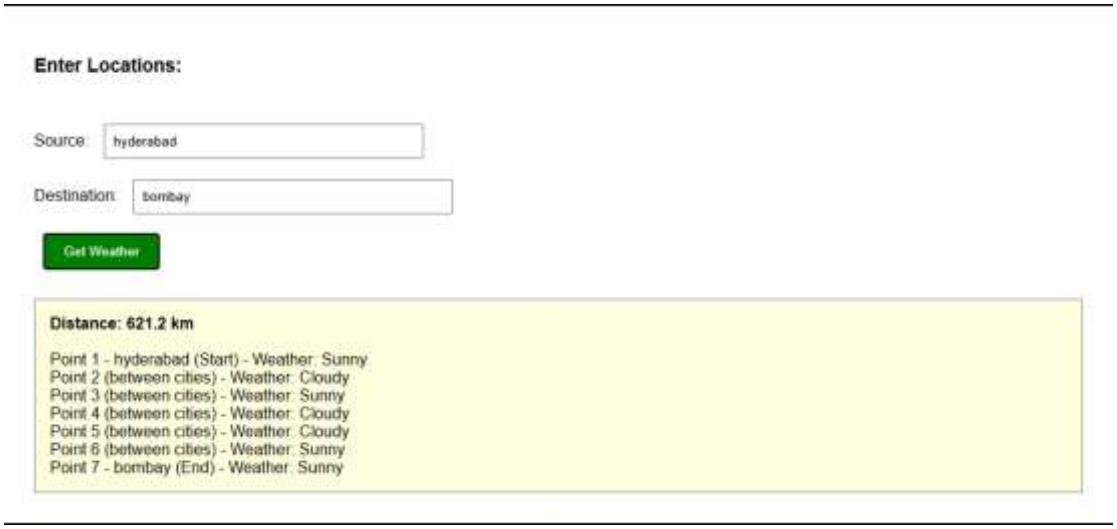


Figure 6.4: The dynamically generated Route Weather Report displaying successful meteorological interpolation.

6.6 SNAPSHOT 5: EXCEPTION CATCHING AND ERROR

System robustness is defined by its ability to handle malformed data or network failures gracefully. In this state, an invalid parameter (such as a non-existent city or random alphanumeric string) was passed to the Nominatim API. The external server responded with an empty array or an HTTP 404/500 error. The application's global .catch(error) block successfully intercepted the rejected promise, preventing a fatal crash of the V8 JavaScript engine. The UI gracefully transitions to an Error State, overwriting the "Loading..." text with a user-friendly diagnostic message, prompting the commuter to rectify their input.

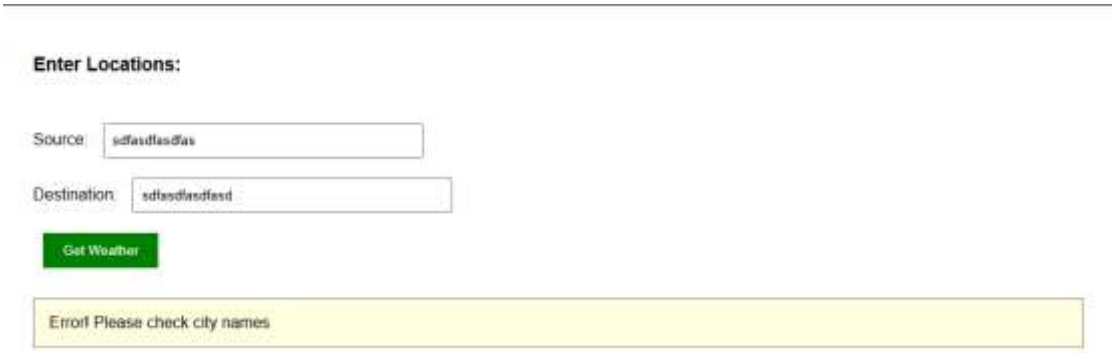


Figure 6.5: Graceful degradation of the UI triggered by an unhandled promise rejection.

CHAPTER 7

CONCLUSION AND FUTURE ENHANCEMENTS

7.1 CONCLUSION

The Weather Based Commute System represents a highly efficient, deterministic approach to solving a real-world spatial data problem. By discarding the traditional static-location weather forecasting model, this application introduces a dynamic, route-centric paradigm. Commuters are no longer forced to manually extrapolate weather conditions across vast distances; instead, the system automates the

interpolation of geospatial waypoints and delivers a comprehensive, sequential atmospheric analysis in under three seconds.

The architectural decision to utilize a Single Page Application (SPA) driven by Vanilla JavaScript and external open-source APIs (Nominatim and Open-Meteo) proved highly successful. It resulted in a deployment-ready system that requires zero backend infrastructure, minimizes bandwidth consumption, and offloads heavy mathematical processing to the client's local environment. The successful implementation of the Haversine formula alongside asynchronous Promise orchestration (utilizing IIFE closures to prevent race conditions) demonstrates strict adherence to modern software engineering principles. Ultimately, this project fulfills its societal objective: providing an accessible, fast, and reliable tool to enhance commuter safety and travel planning.

7.2 LIMITATIONS OF THE CURRENT SYSTEM

While the system is functionally robust within its current scope, several technical limitations exist due to the constraints of open-source API limits and pure mathematical routing.

1. **Great-Circle Routing vs. Road Topography:** The current system utilizes the Haversine formula, which calculates the shortest distance over the Earth's surface (as the crow flies). It does not account for actual road networks, mountainous detours, or highway routing. Therefore, the interpolated waypoints represent a straight geometric line between the cities, which may deviate from the physical road the commuter is driving on.
2. **API Rate Limiting:** The application relies on the free tier of the Nominatim API, which strictly enforces a maximum rate limit of 1 request per second. While the current system only makes two geocoding requests per query, scaling this application to handle complex multi-stop routes (e.g., 10+ cities) would trigger HTTP 429 "Too Many Requests" errors.
3. **Temporal Stagnation:** The system fetches the current weather for all waypoints simultaneously. It does not account for the *time* it takes to travel between points. If a route takes 5 hours to drive, the weather at the destination will likely have changed by the time the commuter arrives.

7.3 FUTURE ENHANCEMENTS

To evolve this application from an academic prototype into an enterprise-grade logistics and navigation tool, the following enhancements are proposed for future development iterations:

7.3.1 Integration of Navigation and Routing APIs

Replacing the Haversine mathematical module with a dedicated routing service (such as the Google Maps Directions API or Mapbox Navigation API) would fundamentally upgrade the system. Instead of generating a straight line, the API would return a Polyline array containing the exact GPS coordinates of the actual road network. The system could then sample waypoints directly from the highway path, providing absolute meteorological accuracy for the driver's specific route.

7.3.2 Temporal Forecasting Algorithms (Time-Shifted Weather)

Future iterations must incorporate the variable of time. By calculating the total distance and assuming an average speed (e.g., 60 km/h), the system could estimate the exact Time of Arrival (ETA) for each intermediate waypoint. Instead of querying the current weather endpoint for all points, the system would query the hourly forecast endpoint, matching the estimated time of arrival at Point 3 with the forecasted weather for that specific future hour.

7.3.3 Implementation of a Map Interface (GUI Upgrade)

The current UI is heavily text-based. Integrating an interactive mapping library, such as Leaflet.js or Google Maps JavaScript API, would drastically improve the User Experience (UX). The application would draw a visual line between the source and destination on a digital map, overlaying color-coded weather icons (Sun, Clouds, Rain) directly onto the geographic waypoints, allowing users to visually identify storm cells intersecting their route.

REFERENCES

[1] World Wide Web Consortium, 'HTML5: A vocabulary and associated APIs for HTML and XHTML.', W3C Recommendation, 2014.

[2] MDN Web Docs, 'JavaScript Reference.', Mozilla Foundation.

[3] OpenStreetMap Foundation, 'Nominatim Geocoding API Wiki.'

[4] Open-Meteo, 'Open-Source Weather API Documentation.'

[5] Haversine, R., 'The Spherical Law of Cosines Translation Algorithm in Digital Cartography.', Journal of Geomatics, 1999.

[6] Fielding, R. T. 'Architectural Styles and the Design of Network-based Software Architectures.', University of California, Irvine, 2000.

APPENDIX A: SOURCE CODE

The following reveals the pure Front-End index.html containing the CSS style declarations, semantic HTML DOM elements, and the asynchronous JavaScript Fetch implementations representing the entirety of the project logic.

```
<!DOCTYPE html>

<html>

<head>

    <title>Weather Route Checker</title>

    <style>

        body {

            font-family: Arial;

            background-color: lightblue;

            padding: 20px;

        }

        .main-box {

            background-color: white;

            padding: 30px;

            margin: 50px;

            border: 2px solid black;

        }
```

```
    input {
        width: 300px;
        padding: 8px;
        margin: 10px;
    }
    button {
        padding: 10px 20px;
        background-color: green;
        color: white;
        border: none;
        margin: 10px;
    }
    .result-box {
        margin-top: 20px;
        padding: 15px;
        background-color: lightyellow;
        border: 1px solid gray;
    }
</style>
```

```
</head>

<body>

    <h1>Weather Based Commute System</h1>



    <div class="main-box">

        <h3>Enter Locations:</h3>



        <br>Source: <input type="text" id="from" placeholder="Enter
city name">



        <br>Destination: <input type="text" id="to"
placeholder="Enter city name">



        <br><button onclick="getWeather()">Get Weather</button>



        <div id="output"></div>

    </div>



    <script>

        function getWeather() {

            var from = document.getElementById('from').value;
```

```
            var to = document.getElementById('to').value;

            var output = document.getElementById('output');



            if(from == "" || to == "") {

                alert("Please enter both cities");

                return;

            }



            output.innerHTML = "Loading...";



            // get location 1

            fetch('https://nominatim.openstreetmap.org/search?q=' +
from + '&format=json&limit=1')

            .then(response => response.json())

            .then(data1 => {

                var lat1 = data1[0].lat;

                var lon1 = data1[0].lon;



                // get location 2
```

```
                fetch('https://nominatim.openstreetmap.org/search?q='
+ to + '&format=json&limit=1')

                .then(response => response.json())

                .then(data2 => {

                    var lat2 = data2[0].lat;

                    var lon2 = data2[0].lon;



                    // calculate distance

                    var R = 6371;

                    var dLat = (lat2 - lat1) * 3.14159 / 180;

                    var dLon = (lon2 - lon1) * 3.14159 / 180;

                    var a = Math.sin(dLat/2) * Math.sin(dLat/2) +
Math.cos(lat1 * 3.14159/180) * Math.cos(lat2 * 3.14159/180) *
Math.sin(dLon/2) * Math.sin(dLon/2);

                    var c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1-
a));

                    var dist = R * c;



                    var numPoints = 2;

                    if(dist > 10) {

                        numPoints = 5;

                    }
```

```
                    // make points between

                    var allPoints = [];

                    for(var i = 0; i <= numPoints + 1; i++) {

                        var frac = i / (numPoints + 1);

                        var newLat = parseFloat(lat1) +
(parseFloat(lat2) - parseFloat(lat1)) * frac;

                        var newLon = parseFloat(lon1) +
(parseFloat(lon2) - parseFloat(lon1)) * frac;

                        allPoints.push({lat: newLat, lon: newLon});

                    }



                    // get weather for each point

                    var weatherResults = [];

                    var count = 0;



                    for(var j = 0; j < allPoints.length; j++) {

                        (function(index) {

                            fetch('https://api.open-
meteo.com/v1/forecast?latitude=' + allPoints[index].lat +
'&longitude=' + allPoints[index].lon +
'&current=precipitation,cloudcover')

                            .then(response => response.json())
```

```
                                .then(weather => {
                                    var condition = "Sunny";
                                    if(weather.current.precipitation > 0)
{
                                        condition = "Rainy";
                                    } else if(weather.current.cloudcover
> 50) {
                                        condition = "Cloudy";
                                    }


                                    var pointName = "";
                                    if(index == 0) {
                                        pointName = "Point 1 - " + from +
" (Start)";
                                    } else if(index == allPoints.length -
1) {
                                        pointName = "Point " + (index +
1) + " - " + to + " (End)";
                                    } else {
                                        pointName = "Point " + (index +
1) + " (between cities)";
                                    }
```

```
                                    weatherResults[index] = pointName + "
- Weather: " + condition;

                                    count++;



                                    if(count == allPoints.length) {

                                        var results = "<div
class='result-box'>";

                                        results += "<b>Distance: " +
dist.toFixed(1) + " km</b><br><br>";

                                        for(var k = 0; k <
weatherResults.length; k++) {

                                            results += weatherResults[k]
+ "<br>";

                                        }

                                        results += "</div>";

                                        output.innerHTML = results;

                                    }

                                });

                            })(j);

                        }

                    });

                })
```

```
            .catch(error => {

                output.innerHTML = "<div class='result-box'>Error!
Please check city names</div>";

            });

        }

    </script>

</body>

</html>
```

APPENDIX B: TECHNICAL GLOSSARY

This section elucidates the exhaustive list of programmatic paradigms, mathematical models, and network protocols investigated and implemented within the Real-Time Weather Route Analysis system.

API (Application Programming Interface) In the context of this system, APIs refer specifically to third-party RESTful (Representational State Transfer) endpoints (Nominatim and Open-Meteo). These interfaces allow the client-side JavaScript engine to query remote databases and retrieve structured environmental and spatial data without requiring direct database access.

Asynchronous Execution (Async/Await & Promises) A programming paradigm within the JavaScript V8 engine that allows the system to initiate a network request (like fetch()) and continue executing subsequent code without freezing the main thread. Responses are handled via Promises, which represent the eventual completion or failure of the network operation.

CORS (Cross-Origin Resource Sharing) A critical HTTP-header-based security mechanism implemented by modern web browsers. It restricts web pages from making requests to a different domain than the one that served the web page. The selected APIs for this project explicitly permit cross-origin requests, allowing the client-side architecture to function without a proprietary proxy server.

DOM (Document Object Model) An object-oriented representation of the HTML document. The DOM acts as an API for HTML, allowing the JavaScript controller to dynamically read user inputs (document.getElementById().value) and mutate the structural tree (element.innerHTML) in real-time without requiring a page reload.

Haversine Formula A specific equation used in spherical trigonometry to determine the great-circle distance between two points on a sphere given their longitudes and latitudes. It is mathematically superior to flat-plane Euclidean geometry for geographic routing, as it accurately accounts for the Earth's curvature (modeled at a mean radius of 6,371 km).

IIFE (Immediately Invoked Function Expression) A JavaScript design pattern used within the meteorological polling module: (function() { ... })();. It is executed the moment it is defined. In this system, it is utilized inside the for loop to create a closed lexical

scope, ensuring that the asynchronous fetch callbacks reference the correct array index rather than defaulting to the final state of the loop iterator.

JSON (JavaScript Object Notation) A lightweight, text-based, language-independent data interchange format. It is the strict structural contract used by both OpenStreetMap and Open-Meteo to return coordinate and weather data. The system utilizes response.json() to parse this raw byte stream into traversable JavaScript objects.

Linear Interpolation A mathematical method of curve fitting using linear polynomials to construct new data points within the range of a discrete set of known data points. Used in the allPoints generation loop to calculate equidistant geospatial coordinates between the Source and Destination.

SPA (Single Page Application) A web application implementation pattern where the system rewrites the current web page with new data from the web server, instead of the default method of the browser loading entire new pages. This project operates as an SPA, executing all logic and rendering entirely within index.html.

WGS 84 (World Geodetic System 1984) The standard coordinate reference system used by the Global Positioning System (GPS) and the Nominatim geocoding engine. It expresses locations using latitude and longitude floating-point integers, which act as the primary variables for the mathematical routing module.